```
import numpy as np


class ForestFireACO:

    def __init__(
        self,
        num_nodes,
        coords,
        fire_risks,
        num_ants=10,
        alpha=1.0,
        beta=2.0,
        rho=0.1,
        Q=10.0,
    ):
        """Args:

        num_nodes: Total sensor nodes in the forest grid.
        coords: Dictionary or list of (x, y) coordinates for each node.
        fire_risks: Array of risk multipliers (higher means urgent fire zone).
        """
        self.num_nodes = num_nodes
        self.coords = coords
        self.fire_risks = fire_risks
        self.num_ants = num_ants
        self.alpha = alpha  # Pheromone importance
        self.beta = beta  # Heuristic importance
        self.rho = rho  # Evaporation rate
        self.Q = Q  # Pheromone deposit factor

        # Initialize matrices
        self.dist_matrix = self._create_dist_matrix()
        self.heuristic_matrix = self._create_heuristic_matrix()
        self.pheromone_matrix = np.ones((num_nodes, num_nodes)) * 0.1

    def _create_dist_matrix(self):
        matrix = np.zeros((self.num_nodes, self.num_nodes))
        for i in range(self.num_nodes):
            for j in range(self.num_nodes):
                if i != j:
                    matrix[i, j] = np.linalg.norm(
                        np.array(self.coords[i]) - np.array(self.coords[j])
                    )
                else:
                    matrix[i, j] = np.inf  # Prevent self-loops
        return matrix

    def _create_heuristic_matrix(self):
        # Heuristic prioritizes short distance AND nodes with high fire risks/urgency
        heuristic = np.zeros((self.num_nodes, self.num_nodes))
        for i in range(self.num_nodes):
            for j in range(self.num_nodes):
                if i != j:
                    # High risk at destination node 'j' increases visibility
                    heuristic[i, j] = self.fire_risks[j] / (
                        self.dist_matrix[i, j] + 1e-5
                    )
        return heuristic

    def _select_next_node(self, current_node, visited):
        probabilities = np.zeros(self.num_nodes)

        for next_node in range(self.num_nodes):
            if next_node not in visited and self.dist_matrix[current_node, next_node] != np.inf:
                tau = self.pheromone_matrix[current_node, next_node] ** self.alpha
                eta = self.heuristic_matrix[current_node, next_node] ** self.beta
                probabilities[next_node] = tau * eta

        total = np.sum(probabilities)
        if total == 0:
            return None  # Dead end or all visited

        probabilities /= total
        return np.random.choice(self.num_nodes, p=probabilities)

    def evaluate_cost(self, path):
        # Calculate total path cost (lower distance + lower risk penalty is better)
        total_dist = 0
        total_risk_reward = 0
        for i in range(len(path) - 1):
```

```
            u, v = path[i], path[i + 1]
            total_dist += self.dist_matrix[u, v]
            total_risk_reward += self.fire_risks[v]

        # Optimization goal: minimize distance, maximize collected critical fire data
        # Cost function penalizes distance and rewards traveling via high-risk nodes
        cost = total_dist / (1.0 + total_risk_reward)
        return cost

    def run(self, source_node, sink_node, max_iterations=50):
        best_path = None
        best_cost = np.inf

        for iteration in range(max_iterations):
            all_paths = []
            all_costs = []

            for ant in range(self.num_ants):
                path = [source_node]
                visited = {source_node}
                current = source_node

                while current != sink_node:
                    next_node = self._select_next_node(current, visited)
                    if next_node is None:
                        break  # Ant trapped, discard path
                    path.append(next_node)
                    visited.add(next_node)
                    current = next_node

                # Only evaluate if the ant successfully reached the base station (sink)
                if current == sink_node:
                    cost = self.evaluate_cost(path)
                    all_paths.append(path)
                    all_costs.append(cost)

                    if cost < best_cost:
                        best_cost = cost
                        best_path = path

            # Pheromone Evaporation
            self.pheromone_matrix *= 1.0 - self.rho

            # Pheromone Deposit
            for path, cost in zip(all_paths, all_costs):
                deposit = self.Q / cost
                for i in range(len(path) - 1):
                    self.pheromone_matrix[path[i], path[i + 1]] += deposit

        return best_path, best_cost


# --- Example Usage Scenario ---
if __name__ == "__main__":
    # Setup 6 sensor nodes scattered in the forest
    # Node 0 is the fire source, Node 5 is the safe Base Station (sink)
    node_coordinates = [
        (0, 0),  # Node 0 (Source)
        (2, 3),  # Node 1
        (5, 1),  # Node 2
        (4, 5),  # Node 3 (High fire risk hot-spot)
        (7, 4),  # Node 4
        (10, 5),  # Node 5 (Sink / Base Station)
    ]

    # Priority values: higher numbers represent active fire lines/high temperature areas
    # Node 3 has a high risk index of 8.0, making it critical to monitor/route through
    node_fire_risks = [1.0, 1.2, 1.1, 8.0, 1.5, 1.0]

    # Initialize ACO
    aco = ForestFireACO(
        num_nodes=6, coords=node_coordinates, fire_risks=node_fire_risks
    )

    # Execute search path from Node 0 to Node 5
    optimal_path, optimal_cost = aco.run(source_node=0, sink_node=5, max_iterations=100)

    print(f"Optimal Network Monitoring Path: {optimal_path}")
    print(f"Calculated Objective Cost: {optimal_cost:.4f}")
```

```
Optimal Network Monitoring Path: [0, 3, 4, 5]
Calculated Objective Cost: 1.1068
```